1. Architecture and data layer

Scope and implementation

SuperChat combines flights, shopping and cinema in one web chat. Domain records come exclusively from the five official mock JSON datasets. Google Gemini interprets messages and proposes responses in LLM mode; no real flight, retail or cinema provider is connected. Reservations are simulated and no payment is collected. The independently selectable rules mode works without a hosted model.

FastAPI serves a self-contained HTML/CSS/JavaScript interface and validated REST endpoints. LLMConversation coordinates GeminiPlanner, CatalogService and StateStore. Pydantic validates source records, structured model responses and API inputs. Repository contracts separate domain operations from JSON loading so Phase 2 can substitute provider adapters. Model-generated plans invoke the same backend services used by REST, rather than recursively calling the application's HTTP endpoints.

Official data and domain operations

The catalog contains 17 flights, 12 products, 8 movies, 3 cinemas and 16 showtimes. Foreign keys join showtimes to films and venues. Official JSON stays immutable; source checksums and schema tests detect unintended changes. Relative dates use 2 November 2026. Flight times are airport-local and duration_min is authoritative.

Flights support origin/destination/date searches; price, stops, departure-time and airline filters; price, duration, departure, stops and airline sorting; details, comparisons and mock reservations. Shopping supports text/category/brand/color/price searches, sorting, details, comparisons, stock and cart totals. Cinema supports title/genre/rating/language/date discovery, cinemas, showtimes, seat availability, prices and mock tickets. The supplied data has no seat map, so reservations specify quantities, not seat numbers.

Storage and consistency

SQLite holds sessions, conversation state, carts and reservations. Inventory is shared across sessions; carts and reservation lists are session-scoped. BEGIN IMMEDIATE transactions prevent overselling. REST request_key retries are idempotent within a session; conflicting reuse is rejected. Cart updates set absolute quantities and validate stock but do not reserve products. Gemini search/detail/comparison responses and REST detail/comparison responses use current seat counts after reservations instead of changing source JSON.

Setup and dependencies

Python 3.11+, FastAPI (MIT), Pydantic (MIT), Uvicorn (BSD-3-Clause) and standard-library SQLite/HTTPS implement the runtime. pytest (MIT) and HTTPX (BSD-3-Clause) support development checks. README links official dependency sources. Run with python run.py; no frontend build or provider SDK is needed. Local .env settings configure the chat mode, key and model; process variables override file values. The selected model is gemini-3.5-flash-lite. A billing-disabled Free Tier project is required for the user's no-paid-API setup; the program cannot verify a key's billing tier.

2. Routing, state and grounded responses

Structured single-call planning

The primary Gemini path classifies each message as Flights, Shopping, Cinema, Mixed, Follow-up, Ambiguous or Out of scope. One generateContent call receives the original message, recent history, per-domain context, saved preferences, reference date and official catalog snapshot. It returns a structured plan containing whitelisted tools and an optional answer draft. This replaced separate planning and response-writing calls to reduce latency and quota use. The backend executes only validated arguments, with at most six operations per turn. Mixed read requests may invoke multiple domains with independent filters; writes must be requested separately.

The planner is instructed to interpret Azerbaijani, Russian and English input, resolve ordinal references against ordered results, preserve filters for follow-ups and ask when destination, date or booking target is missing. These instructions express intended behavior; they do not establish complete language coverage. Provider failures are reported explicitly rather than silently switching to rules. The rules engine remains a separately configured offline mode with narrower English pattern coverage.

Grounding and error handling

Prices, stock, record IDs and reservation totals come from backend records. The draft is produced before execution, then checked against actual result evidence: unknown IDs and unsupported numeric claims reject it. A rejected or absent draft becomes a deterministic summary with verified cards, without another model call. Numeric validation does not establish that every number is attached to the correct record or prove all semantic statements correct; manual review remains necessary. Catalog and history are treated as untrusted data in model instructions.

Network, timeout, quota, missing-key and provider errors produce controlled messages. Known key patterns and the configured key are redacted from exposed provider details. No automatic provider retry, model switch or paid-provider fallback is configured. A configured-key health flag only confirms settings, not successful model access or available quota.

Conversation and preference state

SQLite persists the last active domain, history, per-domain filters, ordered recent results and selected IDs. Returning to a previous topic retains its domain state. Only the last ten compact history turns are sent to planning, alongside separate domain state. Explicit durable preferences are retained independently as bounded verbatim user statements: up to thirty statements, each at most 500 characters. New preference text must occur in the current user message. The planner is instructed to apply relevant preferences unless the current request overrides them; application remains model-dependent.

In Gemini chat, reservation and cart writes first create a persisted pending proposal. Confirm commits after another stock/capacity check; Cancel clears it. A different message invalidates the proposal. Confirm/Cancel for pending writes use no model call. UI cards use REST directly, with a confirmation prompt for reservation. A repeated independent booking request can create another booking; session request-key idempotency is not general chat-message deduplication.

3. Evaluation, limitations and acceptance

Offline and live evidence

The development source tree passes 94 automated tests with one non-failing Starlette/HTTPX deprecation warning. Checks cover source integrity, schemas, foreign keys, filters, money totals, capacity, concurrency, persistence, session isolation, idempotency, context, confirmation/cancellation, current inventory and preference retention. Gemini responses are mocked in automated tests: these tests do not measure live provider quality. The user's local tree previously reported fewer tests, so source-version alignment remains relevant.

The user supplied a live local-server run in LLM mode: all 14 turns across eight official development scenarios passed the smoke checks. Median response time was 1.71 seconds; the maximum was 2.00 seconds. The run covered domain switching, contextual continuations, mixed intent, ambiguity and out-of-scope input. Evidence is the user's pasted terminal output; the JSON report has not been independently inspected. These checks assess expected category labels and minimal result structure, not full factual correctness, preference application or booking behavior. They are a small development sample, not a held-out accuracy estimate or latency guarantee.

The historical rules-router evaluation scored 41/44 (93.18%). Its misses were "I need a computer", "Screenings this weekend" and "Can I travel by plane to Istanbul?", reflecting missing rule vocabulary. This result must not be attributed to Gemini. Live model error analysis requires reviewing actual responses; zero smoke-check failures does not establish zero conversational errors.

Operational and requirement limits

Use one Uvicorn worker: conversation locking is process-local and holds during model requests, so concurrent turns can queue. Session IDs are demo identifiers, not authentication. Hosted-model availability, rate limits and network latency remain external dependencies. Preference updates and correct free-form interpretation are not guaranteed across all inputs. Snapshot answer drafts may race with changing inventory; cards reflect backend results and factual checks are partial.

English-only replies follow the user's explicit project choice and intentionally differ from the brief's requirement to answer in the user's language. Full multilingual conversations, voice input and visual search are not implemented. Round-trip flights are optional and excluded. Real data APIs and production authentication are Phase 2 work. Gemini is the hosted language model, not a real-data provider.

Remaining acceptance

The local server and live routing smoke run are confirmed by user output. Full browser acceptance remains open: cart add/remove/total, reservation Confirm/Cancel and current seats, page refresh restoration, mobile layout and factual review of generated explanations. Rehearse the documented five-minute demo with at least three domain switches; its timing has not yet been measured. These open checks prevent claiming complete Phase 1 acceptance solely from passing tests.

Sources

Organizer brief and five official datasets define requirements and records. Dependency repositories and licences are linked in README. Gemini configuration references: https://ai.google.dev/gemini-api/docs, https://ai.google.dev/gemini-api/docs/billing. No claim of production readiness or independently audited live-answer accuracy is made.